

Java Developer Hiring Assignment (POC) Offline■First Collaborative Text using CRDT Concepts

1. Purpose of This Assignment

This assignment is a Proof of Concept (POC) task designed to evaluate a candidate's real■world engineering skills. The focus is on distributed■system thinking, correctness of logic, and clarity of explanation rather than UI, frameworks, or boilerplate code.

2. Background and Context

In real applications such as collaborative editors, notes apps, or messaging drafts, users may edit data while offline. When devices reconnect, changes may arrive at the server in any order. Naive approaches often break in such scenarios. This POC addresses that challenge.

3. Problem Statement

Build a Java■based system that allows multiple clients to modify a shared text document independently and later synchronize changes without conflicts. The system must remain correct even when operations are received out of order.

4. Core Challenge (Mandatory)

The most important requirement of this POC is handling the DELETE■before■INSERT scenario. If a delete operation arrives before the corresponding insert operation, the system must still apply the delete correctly once the insert arrives.

Example Scenario

- Client A inserts character 'H'
- Client B deletes that character
- Due to network delay, DELETE reaches server first
- INSERT reaches server later

Expected behavior: Character 'H' must never appear in the final document.

5. Technical Requirements

- 1 Use pure Java (standard library only). No Spring Boot, no external CRDT libraries.
- 2 Use an operation■log based approach (do NOT sync full document text).
- 3 Each operation must have a globally unique ID (Lamport timestamp + clientId + sequence).
- 4 INSERT operations must be anchor■based (reference another element ID, not an index).
- 5 DELETE operations must use tombstones (logical deletion).
- 6 Out■of■order delivery must be supported (especially delete■before■insert).
- 7 Document rendering must be deterministic.

6. Suggested Data Model (Guidance)

You are free to design your own data model, but a typical approach may include:

- OpId: Unique identifier for each operation
- Operation: Insert or Delete with metadata
- Element: Represents inserted content with a tombstone flag
- Pending Tombstones: Storage for deletes whose targets are not yet received

7. API Expectations

- 1 POST /op – Accepts an INSERT or DELETE operation in JSON format.
- 2 GET /sync?since=N – Returns all operations after version N (delta sync).
- 3 GET /doc – Returns the current rendered document.
- 4 GET /health – Optional endpoint for server status.
- 5 GET /demo – Optional endpoint to demonstrate delete■before■insert behavior.

8. Demonstration Requirement

Your submission must include a clear demonstration (via endpoint, main method, or test) showing the delete■before■insert scenario. The expected final document output for the demo is: i

9. Deliverables

- 1 Runnable Java source code.
- 2 Instructions to compile and run the application.
- 3 Example requests (curl or equivalent).
- 4 Explanation of how delete■before■insert is handled.

10. Evaluation Criteria

- 1 Correct handling of out■of■order operations.
- 2 Quality and clarity of Java code.
- 3 Understanding of distributed■system challenges.
- 4 Ability to clearly explain design decisions.
- 5 Completeness of the POC.

11. Notes for Candidates

This is a POC, not a production system. Focus on correctness and reasoning. Over■engineering is discouraged. Clear explanations matter as much as code.

Successful completion of this POC is required to proceed with the hiring process.